1. Identify any problems (i.e. compile errors or improper design practice) with the module below:

```
module foo (input [2:0] a,b, output logic [3:0] c);
always_comb
  if (b)
    assign c = a & b;
  else
    assign c = a | b;
endmodule
```

1. **assign statements inside of always block**
2. **width mismatch between LHS and RHS**
3. **b is 3 bits and used in if-condition (this might be OK)**

2. To what value will the following SystemVerilog code set to signal **b** when a = 4'b1011?

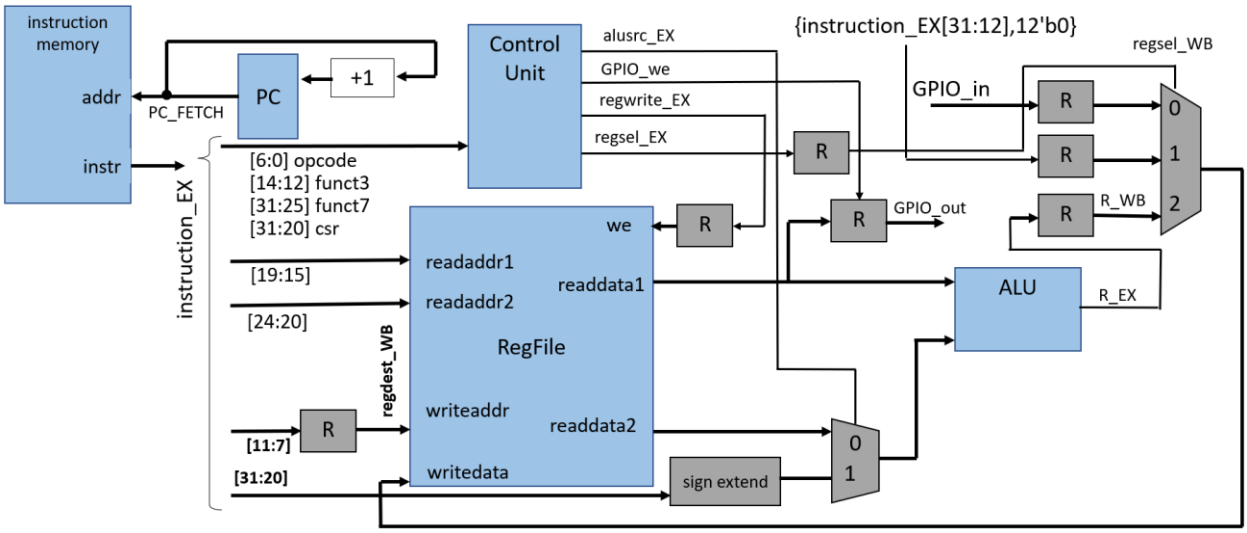
```
module bit_manipulator (input [3:0] a, output [3:0] b);
  assign b = {a[0],a[1],{2{a[2]}};
endmodule
```

1100

3. Write a SystemVerilog module named **parity**. It receives three one-bit inputs, **a**, **b**, and **c**. It produces one one-bit output **y** that is TRUE if an odd number of its inputs are TRUE.

```
module parity(input logic a,b,c,output logic y);
  assign y =  a ^ b ^ c;
endmodule
```

4. Consider the design of the 3-stage pipelined CPU design shown below:



a. (5 points) Write a SystemVerilog snippet to implement the "sign extend" block.

```
assign se = {20{instruction_EX[31]},instruction_EX[31:20]};
```

b. (5 points) Write a SystemVerilog snippet to implement the block that produces the GPIO_out signal. Feel free to use your own signal names when referencing signals that are not explicitly named in the diagram.

```
always_ff @(posedge clk)
  if (GPIO_we) GPIO_out <= readdata1_EX;
```

c. (15 points) In the following table, mark the stage holding each of the instructions listed in the first four rows. Use the following abbreviations: fetch = "F", decode/execute/memory = "E", write back = "W". Assume the instructions appear consecutively and the first instruction is fetched in cycle 0. Give the state of each of the control signals shown in the last five rows in each of the 6 cycles. Use "U" to denote an unknown value and "X" for don't care.

	cycle 0	cycle 1	cycle 2	cycle 3	cycle 4
`lui x2,0`	F	E	W		
`ori x2,x2,0xFF`		F	E	W	
`mult x2,x2,x3`			F	E	W
`cssrw x0,2,x2`				F	E
regwrite_EX	?	1'b1	1'b1	1'b1	1'b0
regwrite_WB	?	?	1'b1	1'b1	1'b1
regsel_EX	?	2'b01	2'b10	2'b10	2'bXX
regsel_WB	?	?	2'b01	2'b10	2'b10
alusrc_EX	?	1'bX	1'b1	1'b0	2'bXX
GPIO_we	?	1'b0	1'b0	1'b0	1'b1

5. What changes to the CPU design in Question 4 would allow the CPU to execute the (fictional) **wrpc** instruction, where behaves as shown below:

Description:	Write the current address of PC+1 into register rd
Operation:	Regs[$rd] = PC_FETCH; advance_pc (1);
Syntax:	wrpc rd

Connect PC_FETCH into input 3 of the “regsel” mux.

6. a. Write a sequence of two RISC-V instructions that would trigger the register file bypass.

lui x2,0

ori x2,x2,0xFF

b. What would happen if the register file lacked write bypassing?

the ori instruction would see the old value of register 2 (prior to the lui instruction)

7. Convert the behavior of the following SystemVerilog module into a truth table:

```
module test_module(input logic [2:0] a, output logic [1:0] y);
    always_comb begin
        if (a[2]) y = 2'b11; else
        if (a[1]) y = 2'b01; else
        y = a[1:0];
    end
endmodule
```

a	y
000	00
001	01
010	01
011	01
100	11
101	11
110	11
111	11

8. a. Write a SystemVerilog module named **minority**. It receives three one-bit inputs, a, b, and c. It produces one one-bit output y that is TRUE if at least two of the inputs are FALSE.

```
module minority(input logic a,b,c,output logic y);
  assign y = !a & !b | !a & !c | !b & !c;
endmodule
```

b. Write a self-checking testbench for this module. You don't need to use a test vector file. Test every possible value of the inputs.

```
module dut;
logic a,b,c,y;
minority dut(.a(a),.b(b),.c(c),.y(y));
initial begin
{a,b,c} = 3'b000; #5;
if (y != 1'b1) $display("error!");
{a,b,c} = 3'b001; #5;
if (y != 1'b1) $display("error!");
{a,b,c} = 3'b010; #5;
if (y != 1'b1) $display("error!");
{a,b,c} = 3'b011; #5;
if (y != 1'b0) $display("error!");
{a,b,c} = 3'b100; #5;
if (y != 1'b1) $display("error!");
{a,b,c} = 3'b101; #5;
if (y != 1'b0) $display("error!");
{a,b,c} = 3'b110; #5;
```

```
if (y != 1'b0) $display("error!");
{a,b,c} = 3'b111; #5;
if (y != 1'b0) $display("error!");
end
endmodule
```

c. (5 points) Write a SystemVerilog module named **majority**. It receives three one-bit inputs, x, y, and z. The produces one one-bit output, a, that is TRUE if at least two of the inputs are TRUE. This module must instantiate the **minority** module and use it to compute its output.

```
module majority(input logic a,b,c, output logic y);
   minority foo (.a(!a),.b(!b),.c(!c),.y(y));
endmodule
```

9. What is the purpose of the sensitivity list in a SystemVerilog “always” statement?

activates the statement when any of the signals in the sensitivity list change value

10. Blocking assignments (=) update the assigned values immediately, while non-blocking assignments (<=) update the assigned values after all other actions associated with the current event have completed.

    How many clock cycles of delay will the code below impose between input “inval” and output “d”?

```
always_ff @(posedge clk) begin
    a = inval;
    b <= a;
    c = b;
    d <= c;
end
```

2

11. Assume the CPU can execute the following instructions: **`add`**, **`sub`**, **`beq`**, **`lw`**, **`sw`**. For which of these instruction is the **RegWrite_EX** signal not asserted?

add, sub, lw

12. Fill in the table for the module below. Use "?" for unknown values.

```
module ddd (input a,clk,output logic b);
  logic b_dly1,b_dly2;
  always_ff @(posedge clk) begin
    b <= b_dly2;
    b_dly2 <= b_dly1;
    b_dly1 <= a;
  end
endmodule
```

cycle	a	b_dly1	b_dly2	b
0	0	?	?	?
1	1	0	?	?
2	0	1	0	?
3	0	0	1	0
4	0	0	0	1

13. Assume the following Verilog module:

```
module aaa (input a,b,output c);
  assign c = ~a & b;
endmodule
```

(a) How many total bits comprise the inputs to module aaa? How many total bits comprise the outputs to module aaa?

2, 1

(b) Fill in the missing values for the test vectors for testing module **aaa** (multiple solutions may be possible):

a	b	expected c
1'b0	1'b0	0
0	1'b1	1'b1
1	1'b0	1'b0
1'b1	1	1'b0

Now assume the following module, which instantiates the module above.

```
module bbb (input a,c,output b);
  aaa my_aaa (.a(c),.b(a),.c(b));
endmodule
```

(c) Fill in the missing values for the test vectors for testing module **bbb** (multiple solutions may be possible):

c	a	expected b
1'b0	1'b0	0
1'b0	1'b1	1
1'b1	1'b0	0
1'b1	1'b1	0

14. Describe any problems with the code below:

```
module ccc (input a,output b);
  always_comb if (a==1'b1) b = 1'b0;
  always_comb if (a==1'b0) b = 1'b1;
endmodule
```

should be “output logic b”

b is double driven

15. Describe the process by which a user-entered HDL design is converted to a form that may be loaded onto an FPGA.

synthesize, place and route, generate bitstream, program FPGA

16. Describe the process by which an FPGA is programmed.

bits are shifted serially into the fpga to configure all luts, rams, etc.